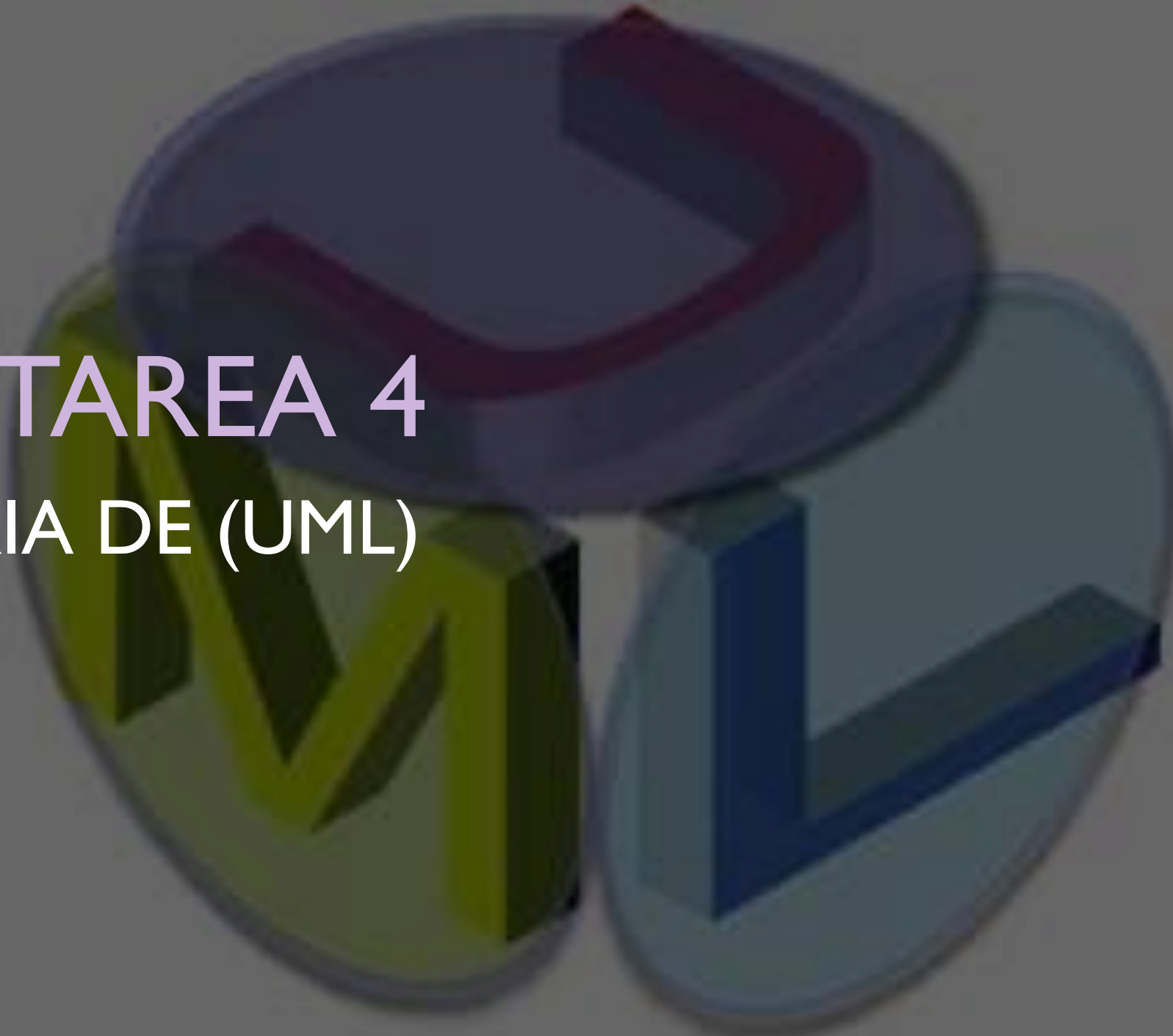
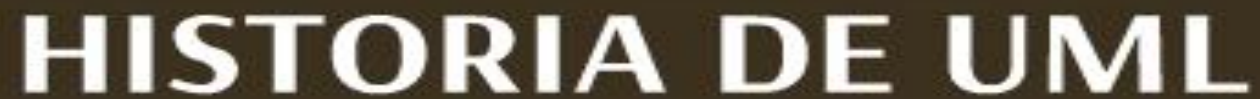




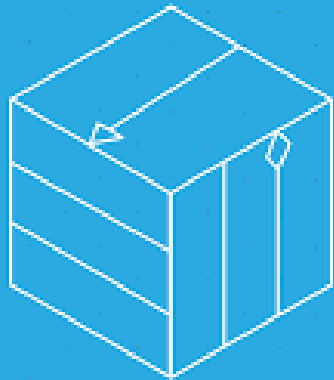
TAREA 4

HISTORIA DE (UML)



La historia del Lenguaje Unificado de Modelado (UML) es el relato de cómo la industria del software pasó de un caos de notaciones a un estándar global para visualizar sistemas complejos

EL ORIGEN: "LA GUERRA DE LOS MÉTODOS"



Uml

A finales de los 80 y principios de los 90, existían decenas de lenguajes de modelado orientados a objetos, cada uno con sus propios símbolos y reglas. Los tres enfoques más influyentes eran:

- Método Booch: Creado por Grady Booch, enfocado en el diseño.
- OMT (Object Modeling Technique): Creado por James Rumbaugh, ideal para el análisis de sistemas.
- OOSE (Object-Oriented Software Engineering): Creado por Ivar Jacobson, introdujo los "casos de uso"

LA UNIFICACIÓN: "LOS TRES AMIGOS"

En 1994, Rumbaugh y Booch comenzaron a combinar sus métodos en la empresa Rational Software. Un año después, Jacobson se unió al equipo, y juntos se hicieron conocidos como "Los Tres Amigos".

- 1995: Lanza el primer borrador llamado Unified Method (UM) versión 0.8.
- 1996: Se añade la "L" al nombre, convirtiéndose formalmente en UML



G. Booch



I. Jacobson



J. Rumbaugh

LA ESTANDARIZACIÓN Y EVOLUCIÓN

Para que UML fuera aceptado universalmente, los autores buscaron el respaldo del Object Management Group (OMG), una organización de estándares sin fines de lucro

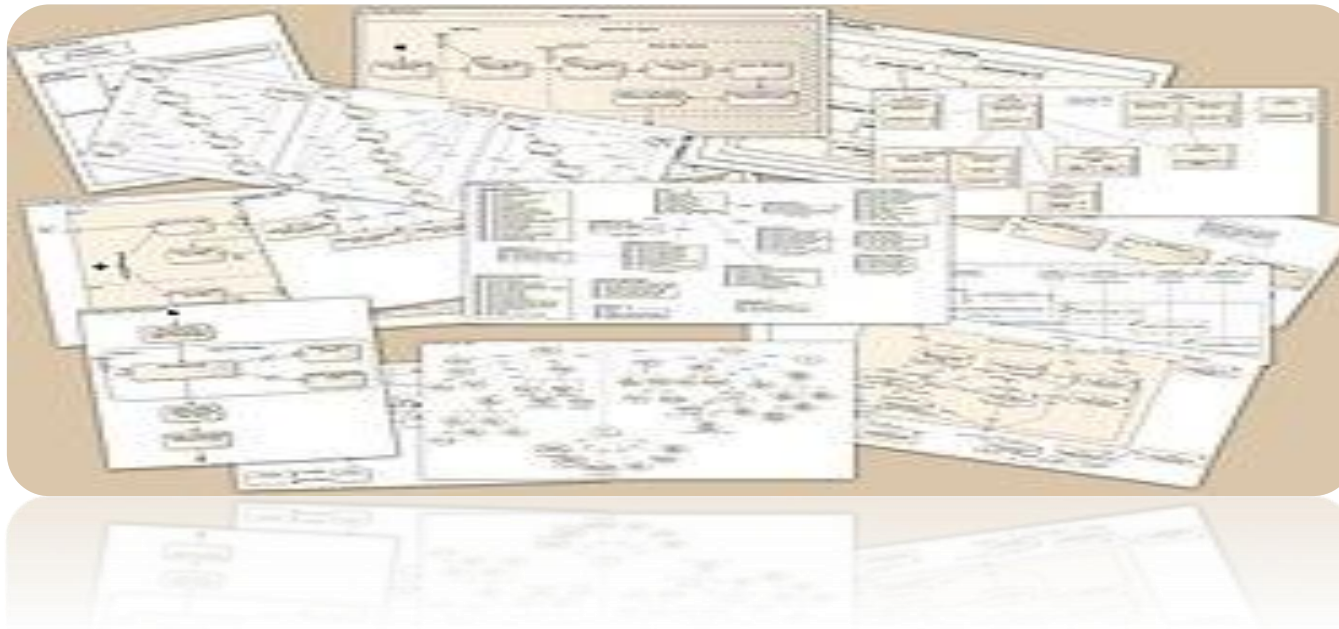
UML 1.1 (1997): Adoptado como el estándar oficial de la industria.

UML 1.x (1997-2005): Se realizaron revisiones menores que consolidaron diagramas básicos como los de clase, objetos, estados y secuencias



EVOLUCIÓN

- UML 2.0 (2005): Una de las actualizaciones más importantes. Reorganizó la arquitectura del lenguaje e introdujo nuevos diagramas como los de estructura compuesta y tiempos.
- UML 2.5.1 (2015 - Presente): Es la versión estable actual, que simplifica la especificación y mejora la documentación para herramientas de modelado moderna



HOY EN DÍA, UML DEFINE 14 TIPOS DE DIAGRAMAS DIVIDIDOS EN DOS CATEGORÍAS: ESTRUCTURALES (ESTÁTICOS) Y DE COMPORTAMIENTO (DINÁMICOS)

A partir de la versión UML 2.5, el estándar reconoce oficialmente 14 tipos de diagramas, los cuales se dividen en dos grandes categorías según lo que intentan representar

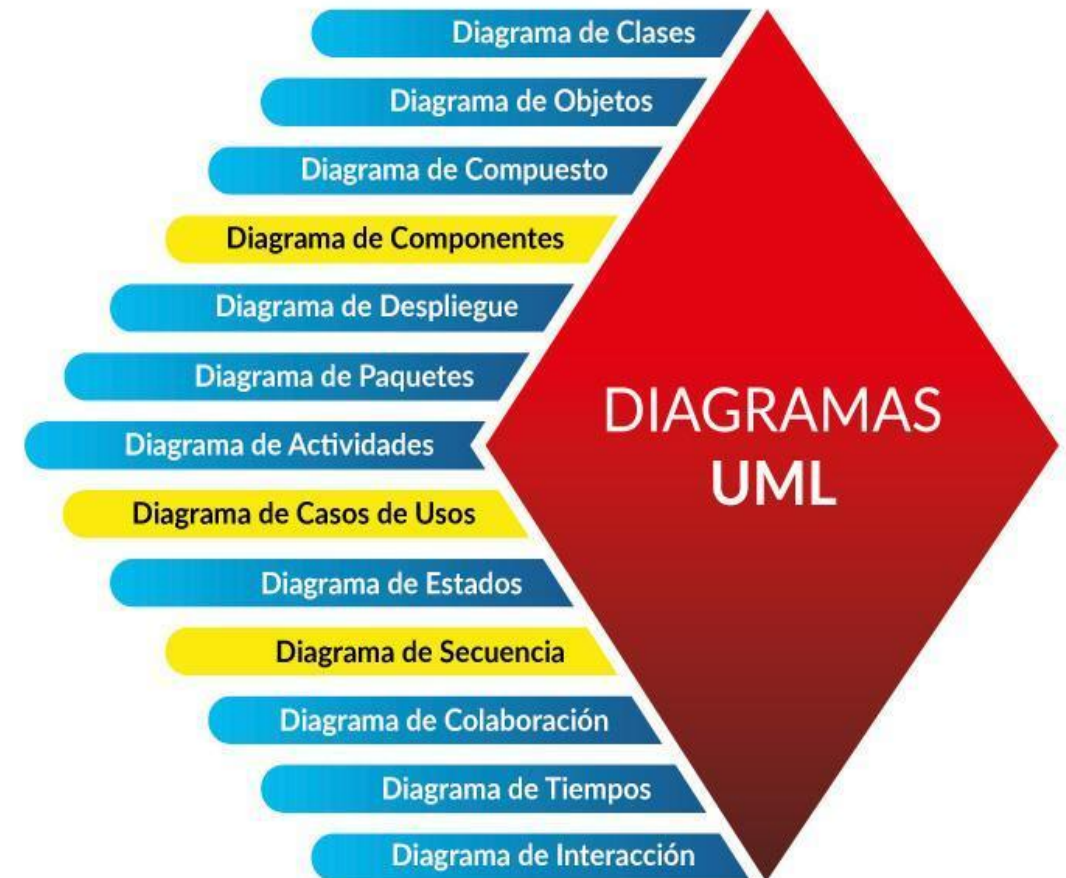


I. DIAGRAMAS ESTRUCTURALES (7 TIPOS)

Diagrama de Clases: El más común; muestra las clases, sus atributos, métodos y las relaciones entre ellas (herencia, agregación, etc.).

Diagrama de Objetos: Una "instantánea" de las instancias de las clases en un momento específico.

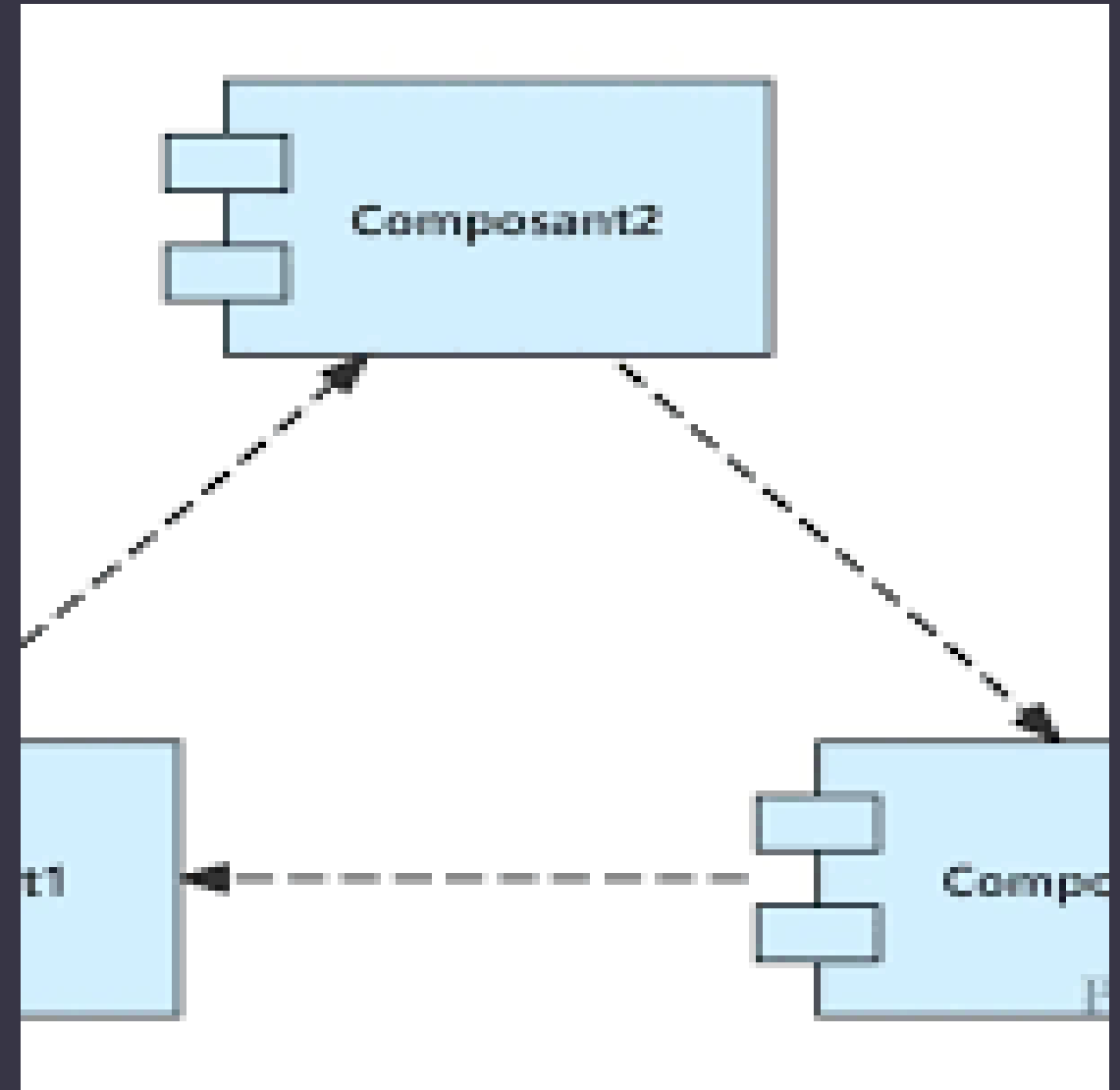
Diagrama de Componentes: Describe cómo se divide el sistema en componentes físicos y las dependencias entre ellos



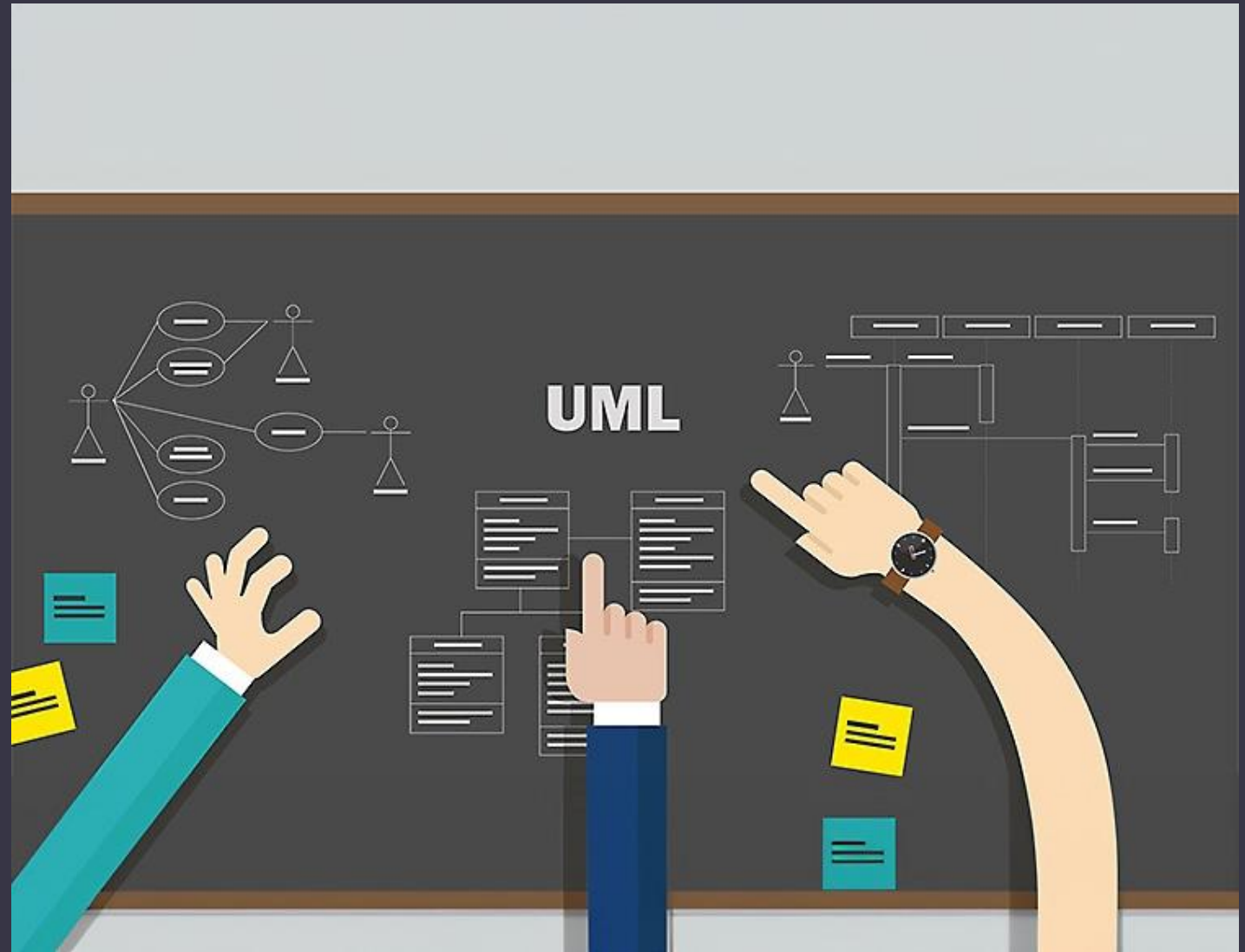
■ **Diagrama de Despliegue (Deployment):** Muestra la arquitectura física del hardware y cómo el software se distribuye en ella.

■ **Diagrama de Paquetes:** Organiza los elementos del modelo en grupos lógicos para mostrar la estructura de alto nivel.

■ **Diagrama de Estructura Compuesta:** Detalla la estructura interna de una clase o componente, incluyendo sus puertos y partes.

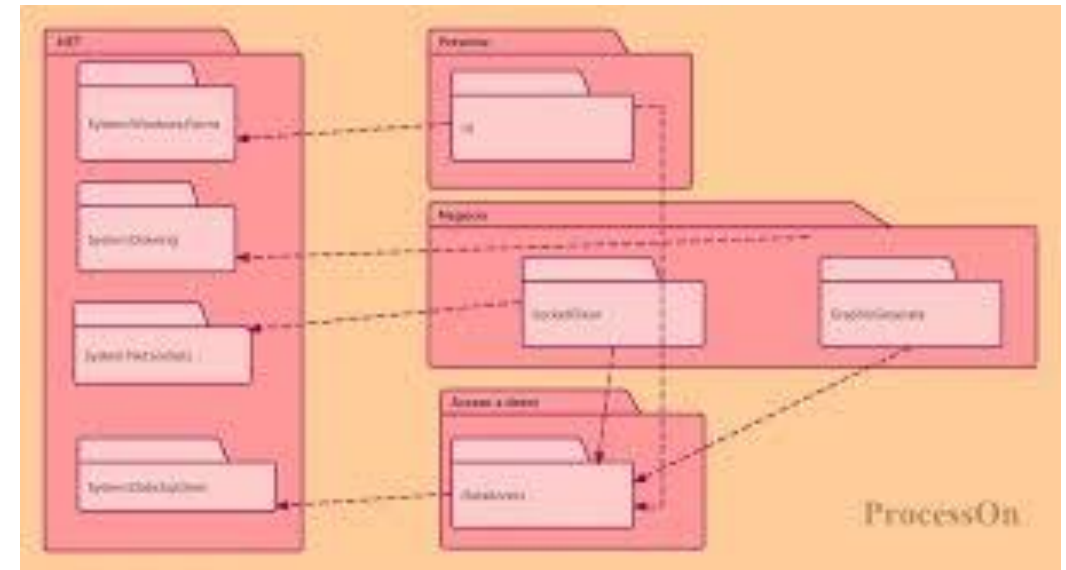


- **Diagrama de Perfiles:** Permite extender UML para dominios específicos (como sistemas embebidos o aeroespaciales) mediante estereotipos



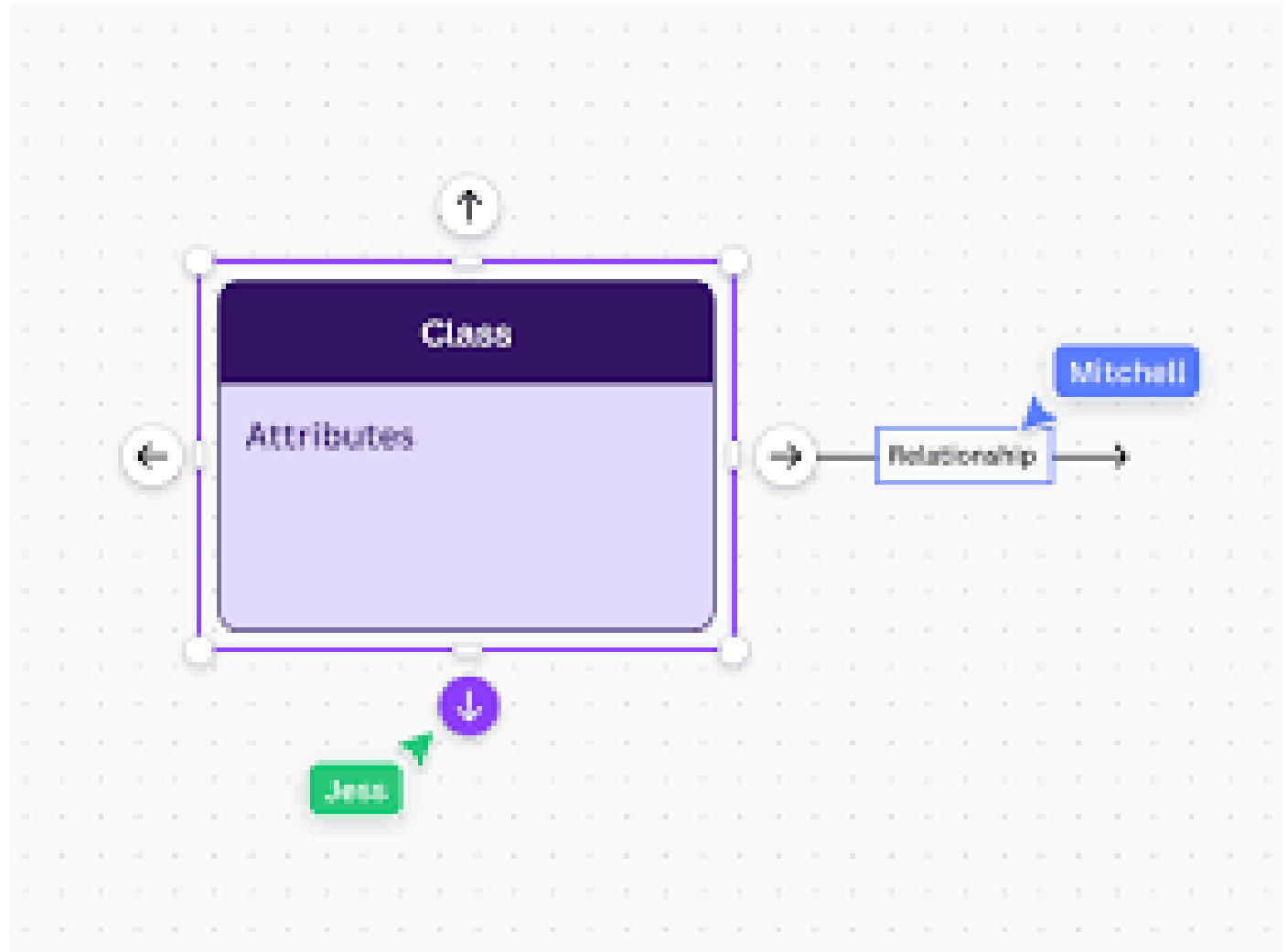
2. DIAGRAMAS DE COMPORTAMIENTO (7 TIPOS)

- Diagrama de Casos de Uso: Describe las funciones del sistema desde el punto de vista del usuario (actor).
- Diagrama de Actividades: Similar a un diagrama de flujo; muestra el flujo de trabajo o procesos paso a paso.
- Diagrama de Máquina de Estados: Representa los diferentes estados por los que pasa un objeto y las transiciones entre ellos

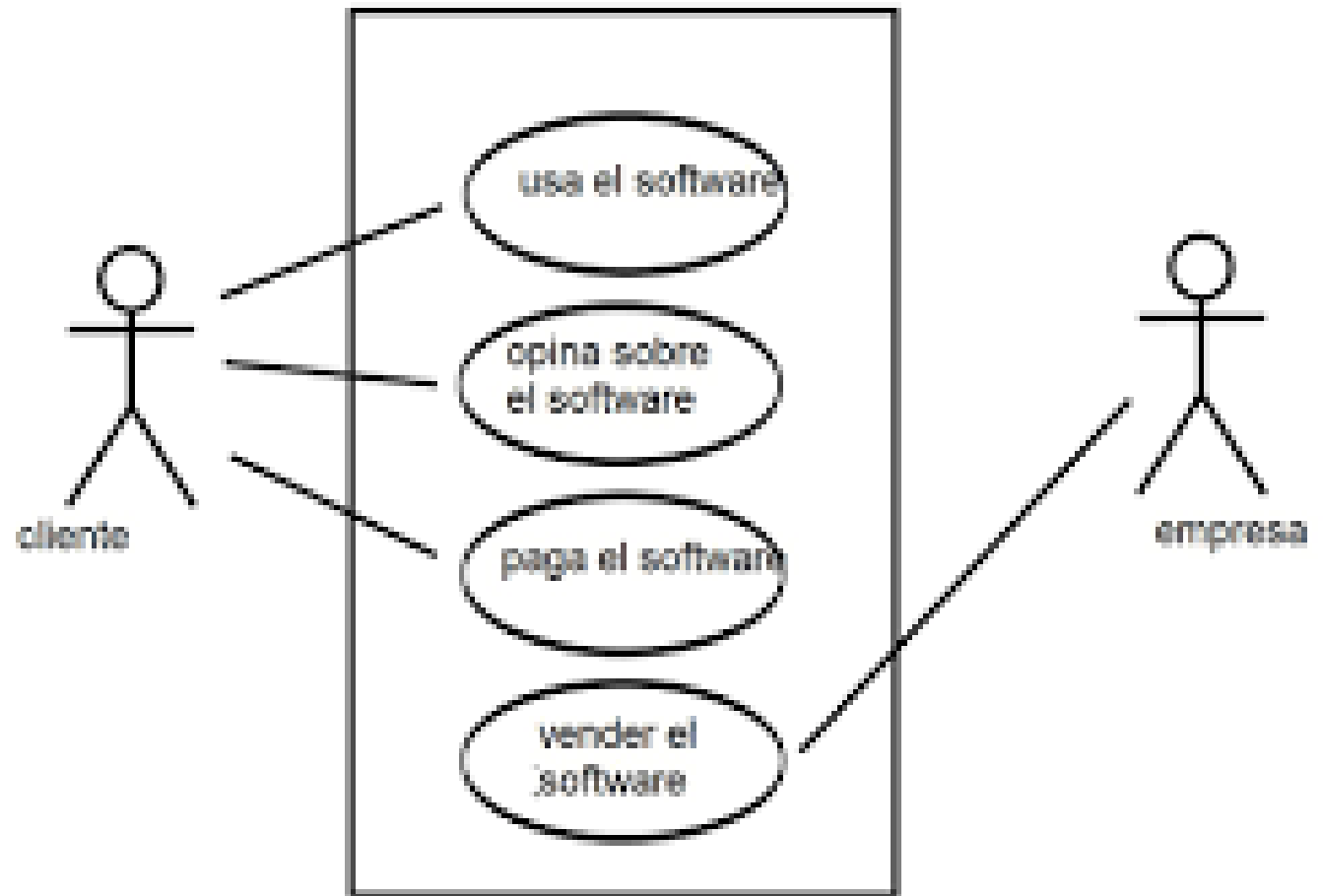


SUBCATEGORÍA: DIAGRAMAS DE INTERACCIÓN

- 11. Diagrama de Secuencia:
Muestra la interacción de
objetos en orden cronológico
(muy usado para lógica de
métodos).
- 12. Diagrama de
Comunicación: Similar al de
secuencia, pero se enfoca en la
organización de los objetos
más que en el tiempo

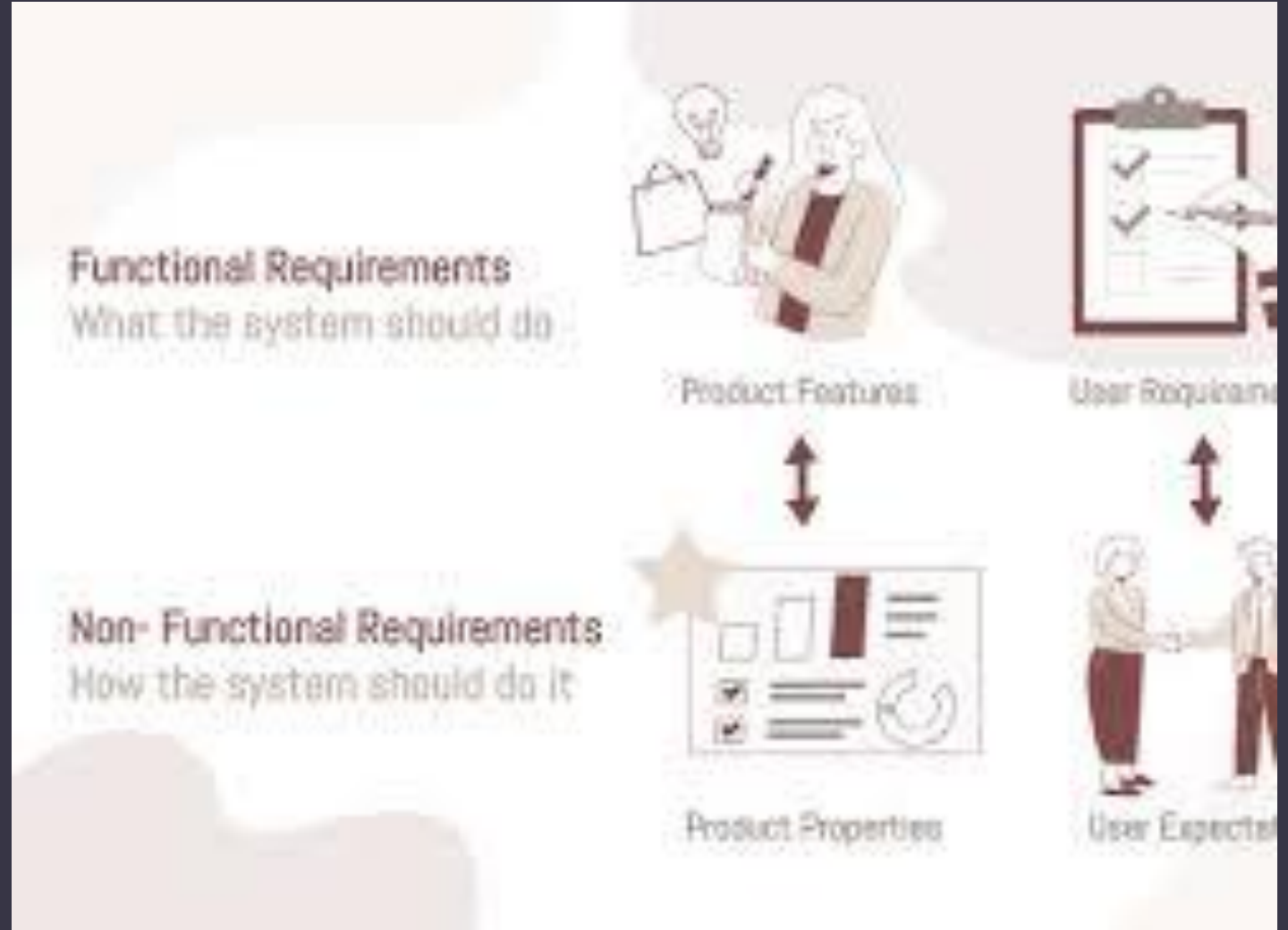


- 13. Diagrama de Tiempos (Timing): Analiza el comportamiento de los objetos en un periodo de tiempo exacto (crucial en sistemas de tiempo real).
- 14. Diagrama de Descripción General de la Interacción: Una mezcla entre el diagrama de actividades y el de secuencia para dar una visión macro de las interacciones



I. REQUERIMIENTOS FUNCIONALES

- Definen las funciones específicas, tareas o comportamientos que el software debe ejecutar para satisfacer las necesidades del usuario. Son la "esencia" del sistema; sin ellos, el programa simplemente no tendría propósito.
- Enfoque: Entradas, procesos y salidas de datos.
- Ejemplos comunes:
- "El sistema debe permitir al usuario iniciar sesión con correo y contraseña".





2. REQUERIMIENTOS NO FUNCIONALES

- También llamados "requisitos de calidad", definen las propiedades, atributos o restricciones del sistema. No describen lo que el sistema hace, sino cómo se comporta (rendimiento, seguridad, usabilidad).
- Enfoque: Experiencia del usuario, estándares técnicos y límites operativos.
- Categorías principales (Modelo FURPS+):
- Usabilidad: Facilidad de aprendizaje y manejo de la interfaz

COMPARATIVA DE REQUERIMIENTOS: FUNCIONALES vs. NO FUNCIONALES

REQUERIMIENTOS FUNCIONALES (El "QUÉ")



¿QUÉ DEFINE?
ACCIONES, TAREAS Y
FUNCIONES DEL SISTEMA



OBJETIVO
FUNCIONALIDAD
ESPECÍFICA A LOGRAR



CRITERIO DE FALLA
EL SISTEMA NO REALIZA
LA TAREA SOLICITADA



ORIGEN
NECESIDADES DIRECTAS
DEL USUARIO



EJEMPLO
"PERMITIR EL REGISTRO DE
NUEVOS CLIENTES"

REQUERIMIENTOS NO FUNCIONALES (El "CÓMO")



¿QUÉ DEFINE?
CÓMO EL SISTEMA CUMPLE LAS
FUNCIONES (ATRIBUTOS DE CALIDAD)



OBJETIVO
RESTRICCIONES O
ESTÁNDARES A SEGUIR



CRITERIO DE FALLA
EL SISTEMA FUNCIONA, PERO
DE FORMA DEFICIENTE



ORIGEN
ARQUITECTURA, SEGURIDAD,
RENDIMIENTO, ESCALABILIDAD



EJEMPLO
"LA BASE DE DATOS DEBE
ESTAR ENCRIPTADA"

MAPA MENTAL RESUMEN

